

Como o código funciona, e por quê

Cada pasta, arquivo, teste e biblioteca do projeto: o que faz, o que lê, o que entrega e por que existe. Para quem vai mexer, revisar ou defender o código.

O QUE É O GIROSUS

GiroSUS é um painel de Power BI para planejamento da ocupação de leitos do SUS em Goiás, feito com dado oficial e mensal das internações (SIH/SUS). Não é uma ferramenta de gestão de leitos em tempo real.

USAMOS UMA FERRAMENTA DE LEITO?

Não. O GiroSUS não usa censo hospitalar, sistema de regulação nem cadastro de leitos. Ele mede o uso do leito a partir das internações pagas pelo SUS. O CNES (leitos cadastrados) só foi usado na exploração.

4

camadas de dados

3

fontes (SIH, CNES,
IBGE)

15

testes automáticos

223

números conferidos

10

tabelas do painel

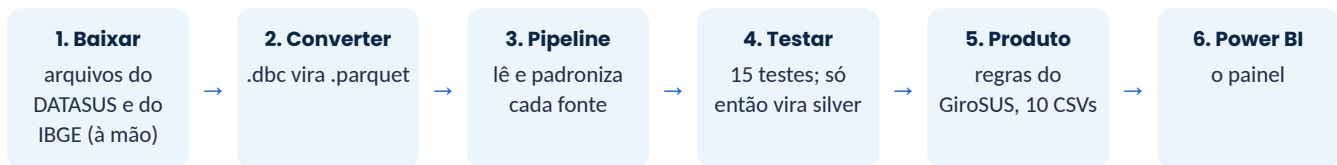
O que tem neste guia

Cada seção começa numa página nova. Nas páginas de arquivo, o quadro **POR QUÊ** explica a decisão por trás do código.

01	O que o código faz, do começo ao fim	O PROJETO EM 1 PÁGINA	3
02	Os 8 princípios do código, e o porquê de cada um	A METODOLOGIA	4
03	Onde fica cada coisa	MAPA DO REPOSITÓRIO	5
04	O caminho do dado e quem chama quem	FLUXO	6
05	raw, staging, silver e gold	CAMADAS DE DADOS	7
06	O que usamos e para quê	BIBLIOTECAS E FERRAMENTAS	8
07	config.py: o painel de controle	SRC/CONFIG.PY	9
08	converter_dbc.py: do .dbc ao .parquet	SRC/CONVERTER_DBC.PY	10
09	extract/: ler os arquivos brutos	SRC/EXTRACT	11
10	transform/: padronizar cada fonte	SRC/TRANSFORM	12
11	pipeline.py e o Prefect	EXECUTAR	13
12	promover_silver.py: a porta da silver	SRC/PROMOVER_SILVER.PY	14
13	tests/: o que cada teste cobre	TESTS	15
14	As cinco conferências do projeto, e o que fica de fora	QUALIDADE	16
15	notebooks/: exploração e análise	NOTEBOOKS	17
16	O script do produto: visão geral	GERAR_MODELO_BI_GIROSUS.PY · 1 DE 3	18
17	As regras e os cálculos, explicados	GERAR_MODELO_BI_GIROSUS.PY · 2 DE 3	19
18	As 10 tabelas, a gravação e a conferência	GERAR_MODELO_BI_GIROSUS.PY · 3 DE 3	20
19	O teste da defasagem	TESTAR_ESTIMATIVA_DEFASAGEM.PY	21
20	Documentação e configuração	ARQUIVOS DE APOIO	22
21	As convenções do código	CONVENÇÕES	23
22	Como rodar tudo, na ordem	NA PRÁTICA	24
23	O que é feito nos dados, do DATASUS ao Power BI	LISTA DOS TRATAMENTOS	25
24	O que ainda pode melhorar	PONTOS DE ATENÇÃO	26
25	Glossário técnico	GLOSSÁRIO	27

O que o código faz, do começo ao fim

O código pega os arquivos públicos do SUS, limpa, confere e transforma em 10 tabelas que o Power BI lê para montar o painel GiroSUS.



As três fontes e o papel de cada uma

FONTE	O QUE É	PAPEL NO PROJETO
SIH/SUS	Sistema de Informações Hospitalares: uma linha por internação paga pelo SUS (AIH)	BASE DO PRODUTO Todos os números do GiroSUS vêm daqui, inclusive o código do hospital (coluna CNES do próprio SIH)
IBGE	População de cada município, por ano	APOIO No painel, só dá o nome dos municípios. Não muda nenhum número
CNES	Cadastro de leitos de cada hospital, por mês	SÓ EXPLORAÇÃO Usado nos notebooks 02 e 05. Não entra no painel

POR QUE SÓ O SIH NOS NÚMEROS

O SIH é a única base que tem, numa linha só, tudo o que o produto mede: datas de entrada e saída, dias internado, diagnóstico, procedimento, hospital, cidade do paciente e valor pago. Com uma base só, não há cruzamento que possa ligar dados errados, e qualquer número do painel pode ser feito a partir dela.

Os dois tipos de código

Ingestão (pasta src/)

Lê, padroniza e testa as três fontes. Não aplica regra de negócio: guarda todas as internações como vieram. Resultado: as bases conferidas em `data/silver`.

Produto (scripts na raiz)

`gerar_modelo_bi_girosus.py` aplica as regras do GiroSUS e grava as tabelas do painel.
`testar_estimativa_defasagem.py` prova os números da defasagem.

POR QUÊ

Separar ingestão e produto permite trocar as regras do painel sem mexer na leitura dos dados, e reaproveitar a mesma base conferida em outras análises.

Os 8 princípios do código, e o porquê de cada um

PRINCÍPIO	O QUE É	POR QUÊ
1. Camadas de dados	O dado passa por raw (bruto), staging (saída do pipeline), silver (conferida) e gold (pronta para o painel). É a “arquitetura medalhão”.	Cada camada tem um papel e só lê a anterior. Se algo der errado, dá para saber em que etapa foi.
2. Uma responsabilidade por arquivo	<code>extract/</code> só lê, <code>transform/</code> só padroniza, <code>pipeline.py</code> só executa na ordem.	Arquivo pequeno é fácil de ler, testar e trocar. Nenhum arquivo faz tudo.
3. Um módulo por fonte	SIH, CNES e IBGE têm um arquivo cada em <code>extract/</code> e em <code>transform/</code> .	Mudar uma fonte não mexe nas outras.
4. Configuração num lugar só	Caminhos, anos e estado ficam em <code>src/config.py</code> .	Para mudar o período ou a pasta, mexe-se em um arquivo, não em dez.
5. Qualidade testada	Os testes conferem a staging antes de ela virar silver.	Ninguém analisa um dado que não passou pela conferência.
6. Regra de negócio fora do pipeline	O pipeline guarda todas as AIHs. Os filtros (IDENT = 1, safra, grupo de doença) ficam no script do produto e no notebook 06.	A mesma base serve para análises diferentes, e a regra do produto fica num lugar fácil de achar.
7. Parquet por dentro, CSV na entrega	raw, staging e silver em Parquet. O gold, que vai para o Power BI, em CSV.	Parquet é compacto e guarda o tipo de cada coluna. CSV é o formato que o time de BI abre no Power BI e no Excel.
8. Reprodutível e com autoconferência	Do .dbc ao painel com poucos comandos, e cada um mostra na tela o que fez e para com ERRO se algo não bater.	Qualquer pessoa refaz o projeto e chega nos mesmos números.

Resumo em uma frase

Cada pedaço do código faz uma coisa só, lê de uma camada e grava na seguinte. A regra de negócio vive num lugar só, e os testes garantem que a base está certa antes de alguém analisar.

Onde fica cada coisa

```
leitos-sus/
├── src/                                # código de ingestão (ETL)
│   ├── config.py                      # caminhos, anos e estado: o painel de controle
│   ├── converter_dbc.py               # .dbc do DATASUS → .parquet em data/raw
│   ├── pipeline.py                    # extrai, padroniza e grava a staging
│   ├── promover_silver.py             # testa a staging e copia para a silver
│   ├── extract/ sih.py cnes.py ibge.py # lê os arquivos brutos
│   ├── transform/ sih.py cnes.py populacao.py # padroniza cada fonte
│   └── orchestration/prefect_flow.py   # o mesmo pipeline no Prefect
├── tests/                             # 15 testes automáticos (pytest)
│   ├── conftest.py                   # carrega a staging para os testes
│   ├── test_sih.py test_cnes.py test_populacao.py
│   └── test_staging_silver.py         # a silver é igual à staging?
├── notebooks/                         # 01 a 03 exploração · 04 consolidação · 05 e 06 análise
├── data/
│   ├── raw/                          sih/ cnes/ ibge/ # arquivos originais convertidos e planilhas
│   ├── staging/                       # saída do pipeline (fora do Git)
│   ├── silver/                        # bases conferidas, lidas pelo produto
│   └── gold/girosus/                  # 10 CSVs + power_query/ (fora do Git)
├── docs/
│   ├── bi/                           GiroSUS_modelagem_bi.pdf · GiroSUS_modelagem_dax.pdf
│   ├── code/                         GiroSUS_como_funciona.pdf # este guia
│   ├── apresentacao/                 GiroSUS_pitch.pdf
│   └── dicionário/                   GiroSUS_dicionario_dados.csv
├── dashboard/                         # pasta reservada (hoje só .gitkeep)
├── gerar_modelo_bi_girosus.py          # cria a camada gold do produto
├── testar_estimativa_defasagem.py      # testa a estimativa dos meses incompletos
├── requirements.txt                   .gitignore .vscode/settings.json README.md
```

Código

`src/` , `tests/` e os dois scripts da raiz.

Dados

`data/` , uma pasta por camada.

Documentação

README na raiz e os PDFs e o dicionário em `docs/` .

Configuração

`requirements.txt` , `.gitignore` e `.vscode/` .

O caminho do dado e quem chama quem



Quem importa o quê e onde grava

ARQUIVO	USA	GRAVA EM
<code>src/converter_dbc.py</code>	config · pysus	<code>data/raw/<fonte>/<ano>/*.parquet</code>
<code>src/pipeline.py</code>	config · extract (sih, cnes, ibge) · transform (sih, cnes, populacao)	<code>data/staging/*.parquet</code>
<code>src/orchestration/prefect_flow.py</code>	config · as funções processar_ do pipeline · prefect	data/staging (pelo pipeline)
<code>src/promover_silver.py</code>	config · pytest (em outro processo) · shutil	<code>data/silver/*.parquet</code>
<code>src/extract/*.py</code>	config (anos e pastas) · pandas	nada: devolvem uma tabela
<code>src/transform/*.py</code>	pandas	nada: devolvem uma tabela
<code>tests/conftest.py</code>	config (caminhos da staging) · pandas	nada: entrega os dados aos testes
<code>tests/test_staging_silver.py</code>	config · pyarrow	nada: compara staging e silver
<code>gerar_modelo_bi_girosus.py</code>	pandas · numpy (lê data/silver e data/raw/sih)	<code>data/gold/girosus/*.csv</code> e <code>power_query/*.pq</code>
<code>testar_estimativa_defasagem.py</code>	as funções do gerar_modelo_bi_girosus.py	nada: só imprime e confere
notebooks 05 e 06	pandas (06 também numpy)	nada

POR QUE EXISTE A STAGING

O pipeline grava na staging e não direto na silver para a silver ser sempre uma versão conferida. A staging funciona como uma área de quarentena: só sai dela o que passou nos testes.

raw, staging, silver e gold

CAMADA	O QUE TEM	QUEM CRIA	QUEM LÊ	NO GIT?
data/raw	SIH (RDGO*.parquet) e CNES (LTGO*.parquet) por ano, todas as colunas como texto; planilhas do IBGE	download à mão + converter_dbc.py	src/extract · gerar_modelo (só VAL_UTI e DIAR_ACOM)	Sim (menos os .dbc)
data/staging	sih, cnes e populacao_multianual.parquet	src/pipeline.py	os testes	Não
data/silver	as mesmas 3 bases, já conferidas	src/promover_silver.py	gerar_modelo · notebooks 05 e 06 · teste staging × silver	Sim
data/gold/girosus	10 tabelas do painel em CSV e 10 consultas .pq	gerar_modelo_bi_girosus.py	Power BI	Não (a fato passa de 250 MB)

O tamanho de cada base na silver

ARQUIVO	LINHAS	COLUNAS	CADA LINHA É
sih_multianual.parquet	2.286.755	23	uma AIH, competências de jan/2021 a jun/2026
cnes_multianual.parquet	198.448	12	um tipo de leito de um hospital em um mês (jan/2021 a jul/2026)
populacao_multianual.parquet	33.422	10	um município em um ano (2021 a 2026)

A IDEIA POR TRÁS

Quanto mais para a direita, mais limpo e mais perto do uso. Ninguém analisa o raw direto, ninguém escreve na silver sem passar pelos testes, e o Power BI só enxerga o gold. Assim um erro fica preso na camada onde nasceu.

POR QUE O GOLD FICA FORA DO GIT

A pasta gold não vai para o Git porque o CSV da fato_internacoes passa de 250 MB, e o GitHub recusa arquivos acima de 100 MB. Cada pessoa gera o seu rodando o script, e o resultado é sempre o mesmo.

O que usamos e para quê

BIBLIOTECA	O QUE É	ONDE USAMOS	POR QUE ESSA
Python 3.12	A linguagem do projeto	tudo	Versão estável, suportada por todas as bibliotecas abaixo
pandas 2.3.3	Tabelas (DataFrames) na memória: ler, filtrar, agrupar, juntar	todo o código e notebooks	O padrão do Python para dados em tabela
numpy	Contas rápidas com vetores de números. Vem instalado junto com o pandas	gerar_modelo (ocupação diária e tendência) · notebook 06	Expandi cada internação em dias sem laço, o que torna o cálculo rápido
pyarrow 25.0.1	Motor que lê e grava Parquet	leitura e gravação de Parquet · teste staging × silver	O pandas usa para Parquet, e o teste compara coluna a coluna
pysus 2.11.2	Biblioteca que lê o formato .dbc do DATASUS	converter_dbc.py · notebooks 01, 02 e 04	O .dbc é um formato compactado antigo que o pandas não lê
openpyxl e xlrd	Leem planilhas do Excel	extract/ibge.py (via pandas)	openpyxl lê .xlsx (2026); xlrd lê .xls (2021 a 2025)
pytest 9.1.1	Ferramenta de testes automáticos	tests/ · promover_silver.py	Roda todos os testes com um comando e mostra quais falharam
prefect 3.8.6	Orquestrador: executa, registra e agenda etapas	orchestration/prefect_flow.py	Opcional; mostra qual etapa falhou e permite agendar
ipykernel (<7)	Liga os notebooks ao Python do .env	notebooks no VS Code	A versão 7 trava a extensão Jupyter do VS Code
duckdb	Banco de dados analítico	está no requirements.txt, mas nenhum arquivo usa hoje	Mantido na lista de propósito

Da biblioteca padrão do Python (não precisa instalar)

MÓDULO	ONDE	PARA QUÊ
<code>pathlib</code>	todo o src, scripts	Montar caminhos de pasta que funcionam no Windows e no Linux
<code>asyncio</code>	converter_dbc.py	O PySUS lê o .dbc de forma assíncrona; o asyncio roda essa leitura
<code>subprocess</code> e <code>sys</code>	promover_silver.py	Rodar o pytest com o mesmo Python do .env e ler se passou
<code>shutil</code>	promover_silver.py	Copiar os arquivos da staging para a silver
<code>time.perf_counter</code>	pipeline.py	Medir quantos segundos cada fonte levou

config.py: o painel de controle

Um arquivo sem funções, só constantes. Todo o código de ingestão importa daqui.

```
# Projeto
PROJECT_ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = PROJECT_ROOT / "data"
RAW_DIR, STAGING_DIR, SILVER_DIR = DATA_DIR / "raw", DATA_DIR / "staging", DATA_DIR / "silver"
# Fontes
SIH_RAW_DIR, CNES_RAW_DIR, IBGE_RAW_DIR = RAW_DIR / "sih", RAW_DIR / "cnes", RAW_DIR / "ibge"
# Período e localidade
ANO_INICIAL, ANO_FINAL = 2021, 2026
ANOS = range(ANO_INICIAL, ANO_FINAL + 1)
UF, CODIGO_UF = "GO", "52"
# Saídas (uma de cada para SIH, CNES e POPULACAO)
SIH_STAGING_PATH = STAGING_DIR / "sih_multianual.parquet"
SIH_SILVER_PATH = SILVER_DIR / "sih_multianual.parquet"
```

CONSTANTE	QUEM USA	PARA QUÊ
<code>PROJECT_ROOT</code>	as outras constantes · <code>promover_silver</code>	Caminho da pasta do projeto, calculado a partir do próprio arquivo
<code>ANOS</code>	<code>extract/sih.py</code> , <code>extract/cnes.py</code> , <code>converter_dbc.py</code>	Quais pastas de ano ler em <code>data/raw</code>
<code>*_RAW_DIR</code>	<code>extract</code> · <code>converter_dbc</code>	Onde estão os arquivos brutos
<code>*_STAGING_PATH</code>	<code>pipeline</code> · <code>confest</code> · <code>promover_silver</code> · teste staging × silver	Onde gravar e o que testar
<code>*_SILVER_PATH</code>	<code>promover_silver</code> · teste staging × silver	Para onde copiar e com o que comparar
<code>UF</code> , <code>CODIGO_UF</code>	referência	O recorte é Goiás; os arquivos já vêm filtrados pelo nome (RDGO, LTGO)

POR QUE `PATH(__FILE__)`

Com `Path(__file__)`, o caminho da pasta `data` sai do lugar do próprio arquivo, e não de onde o terminal está aberto. Por isso o código funciona rodado de qualquer pasta. O script do produto faz o mesmo com a sua própria constante `RAIZ`.

POR QUE CENTRALIZAR

Para mudar o período (por exemplo, incluir 2027), basta ajustar `ANO_FINAL` aqui. Sem este arquivo, os anos e caminhos estariam repetidos em vários lugares e seria fácil esquecer um.

converter_dbc.py: do .dbc ao .parquet

O QUE FAZ	Procura em <code>data/raw/sih/<ano>/</code> e <code>data/raw/cnes/<ano>/</code> todo <code>.dbc</code> (RDGO* e LTGO*) que ainda não tem o <code>.parquet</code> ao lado, e converte com o PySUS. Todas as colunas saem como texto.
COMO RODAR	<code>python -m src.converter_dbc</code>
LÊ	os <code>.dbc</code> baixados à mão do site do DATASUS
ENTREGA	um <code>.parquet</code> com o mesmo nome, ao lado do <code>.dbc</code>
NA TELA	“Nada a converter” ou um OK por arquivo convertido

FUNÇÃO	O QUE FAZ
<code>listar_pendentes()</code>	Percorre os anos de <code>ANOS</code> e junta os <code>.dbc</code> sem <code>.parquet</code> ao lado
<code>converter(dbc)</code>	Abre o <code>.dbc</code> com o PySUS e grava o <code>.parquet</code> (função assíncrona)
<code>converter_dbc()</code>	Chama as duas acima, mostra o progresso e diz o próximo passo

POR QUÊ

O DATASUS publica em `.dbc`, um formato compactado antigo que o pandas não lê. Convertendo uma vez para Parquet, o resto do projeto lê rápido e sem depender do PySUS. As colunas ficam como texto para não mudar nenhum valor na conversão: um código como 0965324 não perde o zero.

POR QUE SÓ OS PENDENTES

Só converte o que falta. Assim, ao entrar um mês novo, o comando leva segundos e nunca sobrescreve um arquivo já conferido.

extract/: ler os arquivos brutos

Regra da pasta: só ler e juntar, sem transformar. Cada arquivo devolve uma tabela com os arquivos mensais empilhados.

extract/sih.py

NOME	O QUE FAZ
<code>COLUNAS_SIH</code>	As 22 colunas lidas do arquivo RD: competência, AIH, tipo (IDENT), hospital (CNES), municípios, datas, dias, UTI, valor, diagnóstico, procedimento, óbito, sexo, idade e gestão
<code>listar_arquivos_sih()</code>	Para cada ano de <code>ANOS</code> , procura <code>data/raw/sih/<ano>/RDGO*.parquet</code> e devolve a lista em ordem
<code>extrair_sih()</code>	Lê cada arquivo só com essas colunas, anota <code>ARQUIVO_ORIGEM</code> (ex.: RDGO2505.dbc) e empilha tudo. Se não achar arquivo, para com <code>FileNotFoundError</code>

extract/cnes.py

NOME	O QUE FAZ
<code>COLUNAS_CNES</code>	9 colunas do arquivo LT: CNES, município, tipo de unidade, tipo e código do leito, quantidades (existentes, SUS, não SUS) e competência
<code>listar_arquivos_cnes()</code>	Procura <code>data/raw/cnes/<ano>/LTGO*.parquet</code>
<code>extrair_cnes()</code>	Mesmo padrão do SIH: lê, anota <code>ARQUIVO_ORIGEM</code> e empilha

extract/ibge.py

NOME	O QUE FAZ
<code>ARQUIVOS_IBGE</code>	Qual planilha e qual aba ler em cada ano: 2021, 2022 (Censo, versão TCU), 2024, 2025 e 2026. Não existe 2023: o IBGE não publicou, e o ano é calculado no transform
<code>extrair_populacao_ano(ano)</code>	Lê a planilha do ano com cabeçalho na 2ª linha e os códigos de UF e município como texto
<code>extrair_ibge()</code>	Devolve um dicionário <code>{{ano: tabela}}</code> com todos os anos

POR QUÊ

Ler só as colunas necessárias deixa a leitura leve. A coluna `ARQUIVO_ORIGEM` permite saber de qual arquivo mensal veio cada linha, o que ajuda a investigar qualquer número estranho.

POR QUE O MESMO MOLDE

SIH e CNES seguem o mesmo molde (`COLUNAS_X` + `listar_arquivos_x()` + `extrair_x()`). Para incluir uma fonte nova, basta copiar o molde.

transform/: padronizar cada fonte

Recebe a tabela do extract e devolve outra, padronizada. Não filtra linhas por regra de negócio: só acerta tipos, nomes e colunas derivadas.

transform/sih.py, 1 função

```
df["DT_INTER"] = pd.to_datetime(df["DT_INTER"], errors="raise").dt.date
```

`transformar_sih(df)` converte a data de internação de texto para data. O resto fica como veio. Com `errors="raise"`, uma data inválida para o pipeline em vez de virar vazio sem ninguém saber.

transform/cnes.py, 1 função

`transformar_cnes(df)` tira o ano e o mês do nome do arquivo (`LTG02401` vira 2024, mês 1) e converte as quantidades de leitos (`QT_EXIST`, `QT_SUS`, `QT_NSUS`) para número.

transform/populacao.py, o mais trabalhoso

FUNÇÃO	O QUE FAZ
<code>_padronizar_estimativa()</code>	Renomeia as colunas da planilha, monta o código de 7 dígitos (UF + município), limpa a população (tira pontos e notas de rodapé como "(1)"), descarta linhas que não são município e marca origem e tipo
<code>_padronizar_2022()</code>	O mesmo para a planilha do Censo 2022, que tem outro nome de coluna
<code>transformar_fontes_oficiais()</code>	Junta os anos publicados: 2021, 2022, 2024, 2025 e 2026
<code>_criar_populacao_2023()</code>	Calcula 2023 pela média de 2022 e 2024 e marca como "Cálculo próprio com dados IBGE"
<code>transformar_populacao()</code>	Junta tudo e ordena por ano e código

POR QUE TÃO POUCO NO SIH

O pipeline guarda os dados quase como vieram, para que qualquer regra possa ser aplicada depois sem perder informação. O que o transform faz é só o que todo mundo precisaria de qualquer jeito: tipo certo e nomes padronizados.

POR QUE A POPULAÇÃO DÁ MAIS TRABALHO

As planilhas do IBGE mudam de formato de um ano para outro (nome de coluna, notas de rodapé, linhas de total). Sem essa limpeza, os anos não se juntam. A marcação de origem deixa claro qual número foi publicado e qual foi calculado.

O "_" no começo do nome indica função interna: só é usada dentro do próprio arquivo.

pipeline.py e o Prefect

src/pipeline.py: a ordem de execução

```
def processar_sih():
    df = transformar_sih(extrair_sih())          # ler e padronizar
    df.to_parquet(SIH_STAGING_PATH, index=False) # gravar
    informar("SIH", df, SIH_STAGING_PATH, inicio) # mostrar na tela

def executar_pipeline():
    STAGING_DIR.mkdir(parents=True, exist_ok=True)
    processar_sih(); processar_cnes(); processar_populacao()
```

O QUE FAZ	Para cada fonte: extrair, transformar, gravar na staging e mostrar linhas, colunas e tempo
COMO RODAR	<code>python -m src.pipeline</code>
NA TELA	SIH 2.286.755 linhas e 23 colunas · CNES 198.448 e 12 · População 33.422 e 10

POR QUE PYTHON -M

O `-m` roda o arquivo como módulo do pacote `src`. Sem ele, os imports `from src...` não funcionam e aparece “No module named src”.

src/orchestration/prefect_flow.py: o mesmo pipeline no Prefect

```
@task(name="Processar SIH")
def tarefa_sih(): processar_sih()
...
@flow(name="pipeline-leitos-sus")
def executar_fluxo(): tarefa_sih(); tarefa_cnes(); tarefa_populacao()
```

Cada `processar_x()` vira uma `@task` (etapa), e `executar_fluxo()` é o `@flow` (fluxo). Roda local, sem servidor: `python -m src.orchestration.prefect_flow`.

POR QUE EXISTEM OS DOIS

O `pipeline.py` é o jeito simples, sem dependência extra. O Prefect é a versão de produção: registra cada execução, mostra qual etapa falhou e permite agendar a atualização mensal. Os dois chamam as mesmas funções, então a lógica não está duplicada.

promover_silver.py: a porta da silver

O QUE FAZ	1) roda os 12 testes de qualidade da staging; 2) só se passarem, copia os 3 arquivos para a silver; 3) roda os 3 testes que conferem se staging e silver ficaram iguais
COMO RODAR	<code>python -m src.promover_silver</code>
SE FALHAR	Nada é copiado e a silver continua como estava. A mensagem diz o que fazer
NA TELA	12 passed · 3 arquivos copiados · 3 passed · “Silver atualizada”

NOME	O QUE FAZ
ARQUIVOS	Pares staging → silver das três bases
<code>rodar_testes(arquivos)</code>	Chama o pytest com o mesmo Python que está rodando (<code>sys.executable</code>) e devolve se passou
<code>promover_silver()</code>	Confere se a staging existe, testa, copia (avisa se o arquivo estiver aberto em outro programa) e confere de novo

POR QUÊ

A silver é a base que o produto e as análises leem. Se um mês chegar com defeito (meses faltando, códigos quebrados, datas inválidas), ele fica preso na staging e não contamina o painel.

POR QUE SUBPROCESS

O pytest é chamado em outro processo com `sys.executable` para garantir que é o Python do `.venv`, com as mesmas bibliotecas, e não outro Python instalado na máquina.

tests/: o que cada teste cobre

15 testes automáticos com pytest, que leem a **staging**: 12 de **qualidade** (o dado faz sentido?) e 3 de **regressão** (a silver mudou sem ninguém saber?). Rodam com `python -m pytest`. Se só os de regressão falharem, a staging é mais nova que a silver: rode `python -m src.promover_silver`.

conftest.py: as fixtures

Carrega cada base da staging uma vez só (`scope="session"`) e entrega aos testes pelos nomes `df_sih`, `df_cnes` e `df_populacao`. Um teste que pede `df_sih` recebe a tabela pronta. Assim o arquivo grande é lido uma vez, e não 15.

ARQUIVO	TESTE	O QUE GARANTE	POR QUE IMPORTA
test_sih.py	cobertura_temporal_sih	66 competências, de 2021-01 a 2026-06	Nenhum mês faltando ou sobrando
	codigos_municipais_sih_validos	MUNIC_RES e MUNIC_MOV com 6 dígitos	O código liga com o nome do município
	colunas_criticas_sih_sem_nulos	Competência, AIH, CNES, municípios, datas e arquivo de origem sem vazio	Sem essas colunas não há leito-dia, hospital nem cidade
	datas_sih_validas	DT_INTER e DT_SAIDA viram data válida	O leito-dia e a ocupação diária dependem das datas
test_cnes.py	chave_cnes_sem_duplicados	CNES + tipo + código do leito + competência é único	Não contar o mesmo leito duas vezes
	quantidade_leitos_consistente	existentes = SUS + não SUS	As quantidades fecham entre si
	cobertura_temporal_cnes	67 competências, de 2021-01 a 2026-07	Nenhum mês faltando
	colunas_criticas_cnes_sem_nulos	Colunas-chave sem vazio	Toda linha identificável
test_populacao.py	chave_populacao_sem_duplicados	Município + ano é único	Um valor por município e ano
	codigos_ibge_validos	Código com 7 dígitos	Liga com os códigos das outras bases
	cobertura_temporal_populacao	Anos 2021 a 2026	Nenhum ano faltando
	proveniencia_populacao_2023	2023: 5.570 municípios, marcado como cálculo próprio	Deixar claro que 2023 não foi publicado pelo IBGE
test_staging_silver.py	sih / cnes / populacao_staging_igual_silver	Mesmas linhas, mesmo esquema e mesmos valores, coluna a coluna (pyarrow)	Detecta se a silver mudou sem passar pela promoção

As cinco conferências do projeto, e o que fica de fora

Os testes do pytest conferem as bases. O produto tem conferências próprias, dentro dos scripts, que param com ERRO se algo não bater.

#	CONFERÊNCIA	ONDE	O QUE COBRE
1	Qualidade da staging	tests/ (12 testes)	Meses completos, códigos no formato certo, colunas-chave sem vazio, chaves sem duplicado
2	Staging igual à silver	tests/test_staging_silver.py (3)	A silver é exatamente a staging que passou nos testes
3	Estrutura do gold	<code>conferir_saida()</code> no <code>gerar_modelo</code>	As 10 tabelas existem, têm todas as colunas, nenhuma célula vazia , não há <code>nome_hospital</code> e não sobrou <code>.parquet</code> antigo
4	Números do painel	<code>calcular_numeros_pdf()</code> no <code>gerar_modelo</code>	Recalcula 223 números do <code>GiroSUS_modelagem_bi.pdf</code> a partir das tabelas geradas, do mesmo jeito que as medidas DAX calculam
5	Números da defasagem	<code>testar_estimativa_defasagem.py</code>	Os 9 números do slide “Como lidamos com a defasagem”

O que não tem teste automático

- **As funções uma a uma.** Os testes conferem o resultado (os dados), não cada função com dados de exemplo. Se uma função mudar e o dado continuar certo, nada falha; se o dado mudar, as conferências 1 a 4 pegam.
- **Os notebooks.** São análise e exploração. O notebook 06 chega aos mesmos números do script, mas isso é conferido rodando os dois, não por teste.
- **O Power BI.** As medidas DAX podem ser conferidas à mão contra o “resultado esperado” de cada medida no `GiroSUS_modelagem_dax.pdf`.

POR QUE CONFERIR OS 223 NÚMEROS

Os números do painel já foram apresentados ao professor e ao cliente. A conferência 4 garante que qualquer mudança no código que altere um desses números seja vista na hora, em vez de aparecer só no Power BI.

notebooks/: exploração e análise

NOTEBOOK	PARA QUE SERVE	LÊ	OBSERVAÇÃO
01_exploracao_sih	Conhecer o SIH: tamanho, colunas, vazios, AIHs repetidas, permanência, UTI, diagnósticos, hospitais e municípios	.dbc (PySUS)	Precisa dos .dbc, que não vão para o Git
02_exploracao_cnes	Conhecer o cadastro de leitos: consistência das quantidades, leitos SUS, tipos de leito	.dbc (PySUS)	Idem
03_exploracao_ibge	Conhecer as planilhas de população: cabeçalho, códigos, linhas que não são município	data/raw/ibge	Roda normal
04_consolidacao_multianual	Primeira versão da consolidação 2021 a 2026, que hoje o pipeline faz	.dbc e planilhas	A última célula grava na silver: não rodar sem necessidade
05_analise_integrada	Cruza SIH, CNES e população: demanda por habitante, fluxo entre municípios, capacidade × demanda	data/silver	Anterior ao GiroSUS; não alimenta o painel
06_girosus_ocupacao_leitos	A análise do GiroSUS: as perguntas P1 a P34 do painel na safra 2025, em três níveis, e as curiosidades de pagamento. A sazonalidade (P35 a P40) e a tendência saem do script	data/silver (+ data/raw para VAL_UTI)	Mesmos números do script e do PDF

Regra de ouro

Os notebooks de análise (05 e 06) leem só a silver. Assim todo mundo analisa a mesma versão conferida.

Caminhos relativos

Os notebooks usam `../data/silver`. O `.vscode/settings.json` faz cada notebook rodar a partir da própria pasta.

POR QUE NUMERADOS

A ordem dos números é a ordem do raciocínio: primeiro conhecer cada fonte (01 a 03), depois juntar (04), depois analisar (05 e 06). Quem lê os notebooks em sequência entende como o projeto chegou ao produto.

Estrutura do notebook 06

Configuração → carregamento → regras e safra (funil de 467.902 para 457.403 internações) → colunas do produto (leito-dia, grupo, tempo típico) → os três níveis de perguntas → simulação de ganho → curiosidades de pagamento → resumo para a venda → conclusão e limites.

O script do produto: visão geral

Fica na raiz porque não faz parte da ingestão: é a camada do produto. Lê a silver, aplica as regras do GiroSUS e grava as 10 tabelas do Power BI. Leva cerca de 1 minuto e meio.

COMO RODAR	<code>python gerar_modelo_bi_girosus.py</code>
LÊ	<code>data/silver/sih_multianual.parquet</code> (a base) · <code>data/silver/populacao_multianual.parquet</code> (só nomes de municípios) · <code>data/raw/sih/*/RDG0*.parquet</code> (só VAL_UTI e DIAR_ACOM)
ENTREGA	10 CSVs em <code>data/gold/girosus/</code> e 10 consultas <code>.pq</code> em <code>power_query/</code>
FIM ESPERADO	10 linhas OK e “Todos os 223 numeros conferidos batem com o PDF”

O que acontece, na ordem (função `main()`)

PASSO	FUNÇÃO	O QUE FAZ
1	<code>carregar_sih()</code>	Lê a silver e fica só com AIH nova (IDENT = 1)
2	<code>carregar_complementos_raw()</code>	Lê VAL_UTI e DIAR_ACOM do bruto. Sem os arquivos, para com ERRO
3	<code>montar_fato_internacoes()</code>	Monta a tabela de internações de todas as safras , com as colunas do produto
4	filtro da safra	Fica só com a safra 2025 (<code>SAFRA_PADRAO</code>)
5	<code>montar_fato_ocupacao()</code>	Com todas as safras, conta quem estava internado em cada dia; fica só com os dias de 2025
6	dimensões	tempo, diagnóstico, procedimento, hospital e município
7	<code>montar_sazonalidade()</code> e <code>montar_tendencia()</code>	As duas tabelas de apoio
8	limpeza	Apaga <code>.parquet</code> antigos da pasta gold (a saída hoje é só CSV)
9	<code>gravar()</code> e <code>gravar_power_query()</code>	Grava cada tabela em CSV e a consulta <code>.pq</code> correspondente
10	<code>conferir_saida()</code>	Confere estrutura e vazios
11	<code>calcular_numeros_pdf()</code>	Recalcula e confere os 223 números

POR QUE MONTAR TODAS AS SAFRAS ANTES DE FILTRAR

O tempo típico é calculado dentro de cada safra; a ocupação de 2025 precisa das AIHs cobradas em 2024 e 2026 (quem entrou em dezembro de 2025 só é cobrado em 2026); e a tendência olha 2022 a 2025. Por isso o script monta tudo primeiro e só depois recorta 2025.

As regras e os cálculos, explicados

REGRA	COMO É CALCULADA	POR QUÊ
Só AIH nova	IDENT = 1	A AIH de continuação (IDENT = 5) é a mesma internação cobrada em partes. Contar as duas duplicaria o paciente
Safra	Ano da competência (ANO_CMPT). O painel usa 2025	É o ano em que o hospital apresentou a conta ao SUS, que é como o DATASUS organiza os arquivos mensais
Leito-dia	DIAS_PERM da AIH	Mede a ocupação: um paciente de 30 dias pesa como dez de 3 dias
Tempo típico	Mediana dos dias do mesmo procedimento (PROC_REA) na mesma safra	A mediana não é puxada por poucos casos muito longos
Dias acima do típico	dias internado menos o tempo típico, nunca abaixo de 0	É o espaço de melhoria que o simulador usa
Grupo de doença	Primeira letra do CID (J = Respiratório, I = Circulatório, S e T = Lesões e traumas). Na letra D: D00 a D48 é Câncer e D50 a D89 é Sangue	O capítulo da CID-10 é a forma oficial de agrupar diagnósticos
Faixas de duração	0 dia, 1 a 3, 4 a 7, 8 a 15, 16 a 30 e mais de 30 dias, com <code>ordem_faixa</code> para ordenar	Mostra quem fica pouco e quem fica muito
Mora fora	município de residência diferente do de atendimento	Mede quanto uma cidade depende de outra

A ocupação diária, passo a passo

Regra: o paciente está internado no dia **d** se entrou até **d** e ainda não saiu (`data_internacao ≤ d < data_saida`).

1. Cada internação é cortada para caber em 2025 (quem entrou antes começa em 01/01; quem saiu depois termina em 31/12).
2. Com o numpy, cada internação vira uma linha por dia internado, sem laço (uma internação de 5 dias vira 5 linhas).
3. Conta, por hospital, dia e grupo: **internados**. As **entradas** são contadas pela data de internação e as **altas** pela data de saída.

Sazonalidade

Média de internados por dia em cada mês de 2025, por grupo e para a rede toda ("Total da rede"). Dias sem paciente contam como 0. Compara cada mês com a média do ano e marca o pico e o vale.

Tendência

A mesma regra da ocupação, de 2022 a 2025: média das 12 médias mensais de cada ano. 2021 fica de fora porque quem já estava internado em jan/2021 entrou antes da base, e o começo de 2021 sairia baixo demais.

As 10 tabelas, a gravação e a conferência

TABELA	CADA LINHA É	LINHAS
fato_internacoes	uma internação (AIH nova) da safra 2025, 25 colunas	457.403
fato_ocupacao_diaria	um hospital, num dia de 2025, num grupo de doença	392.172
dim_tempo	um dia, de 17/05/2024 (1ª internação da safra 2025) a 31/12/2025	594
dim_diagnostico	um CID, com capítulo e grupo de doença	6.114
dim_procedimento	um procedimento, com diagnóstico mais comum e tempo típico	1.346
dim_hospital	um hospital: código CNES, município e nome do município	260
dim_municipio_residencia e _atendimento	um município (a mesma tabela, uma para cada lado da relação)	1.241
sazonalidade_mensal	um mês de 2025 num grupo de doença, mais o Total da rede (apoio)	252
tendencia_anual	um ano, de 2022 a 2025 (apoio)	4

Detalhes de gravação e por que existem

DETALHE	POR QUÊ
CSV com separador ; e decimal , , em UTF-8 com BOM	É o padrão brasileiro: o Excel e o Power BI em português abrem direto, com acentos corretos
Códigos com zero à esquerda (zfill)	CNES e município são códigos, não números: 0965324 não pode virar 965324, senão não liga com as outras tabelas
Consulta .pq por tabela	Pronta para colar no Power BI: já lê o ; e o decimal , e já deixa os códigos como Texto
Sem nome_hospital	O SIH só tem o código CNES do hospital. A coluna sairia vazia, então ela não existe e a conferência para se ela aparecer
Nomes de procedimento preservados	Se alguém preencher à mão um nome em dim_procedimento.csv , ler_nomes_manuais() mantém na próxima execução. Os que não têm nome aparecem como "Procedimento <código>"
Arquivo aberto no Excel	O script avisa que precisa fechar, em vez de travar com um erro difícil de ler

COMO FUNCIONA A CONFERÊNCIA DOS 223 NÚMEROS

Cada número do GiroSUS_modelagem_bi.pdf tem um valor esperado e uma tolerância de arredondamento. Se todos batem, aparece "Todos os 223 numeros conferidos batem com o PDF". Se algum diverge, o script lista qual e para com ERRO. Se a base não tiver as competências de 2024 e 2026, os números da ocupação aparecem como PULADO, com um aviso.

O teste da defasagem

O SIH chega com 1 a 2 meses de atraso: o hospital apresenta a conta no mês da alta ou nos meses seguintes. Este script mede esse atraso e prova, com os dados de 2025, que dá para estimar os meses incompletos.

COMO RODAR	<code>python testar_estimativa_defasagem.py</code>
USA	as mesmas funções do <code>gerar_modelo_bi_girosus.py</code> (importa o script), com a mesma regra de AIH nova
FIM ESPERADO	“OK: os 9 números do slide ‘Como lidamos com a defasagem’ batem”

ETAPA	O QUE FAZ	RESULTADO
1. Atraso da cobrança	Das altas de 2024, quanto foi cobrado no mesmo mês da alta e em até 1, 2 e 3 meses	65,1% no mesmo mês · 99,4% em até 3 meses
2. Estimativa de internações	Para cada mês de 2025, “volta no tempo”, olha só o que já tinha chegado e completa com a curva de atraso dos 12 meses de altas já fechadas	Mês mais recente: erro de 8,4% (contra 29,4% olhando só o que chegou) · um mês antes: 2,7% (contra 12,5%)
3. Estimativa de custo	Custo do mês = leitos-dia estimados × R\$ por leito-dia dos últimos 3 meses fechados, por grupo de doença	Mês mais recente: erro de 4,8% (contra 32,2%) · um mês antes: 4,0%

POR QUÊ

O painel é de planejamento e a base tem atraso. Sem prova, “dá para estimar o mês incompleto” seria só uma promessa. O script refaz o teste com dado real e para com ERRO se algum dos 9 números mudar.

POR QUE MÊS DA ALTA × COMPETÊNCIA

O mês é o mês da alta (data de saída), e a cobrança é a competência. A diferença entre os dois é justamente o atraso que se quer medir.

Documentação e configuração

ARQUIVO	PARA QUE SERVE
<code>README.md</code>	Porta de entrada: o produto, onde está cada coisa, como rodar, fluxo, regras, modelo do Power BI, dicionário, notebooks, testes, como atualizar, solução de problemas e status
<code>docs/bi/GiroSUS_modelagem_bi.pdf</code>	O material de negócio do painel: cada gráfico, a pergunta que responde e os números. Auditado pelo cliente
<code>docs/bi/GiroSUS_modelagem_dax.pdf</code>	Só DAX: todas as medidas, organizadas pela página e pela pergunta do <code>modelagem_bi</code> , com o resultado esperado
<code>docs/code/GiroSUS_como_funciona.pdf</code>	Este guia: como o código funciona e por quê
<code>docs/apresentacao/GiroSUS_pitch.pdf</code>	O pitch curto do produto, em 2 páginas
<code>docs/dicionário/GiroSUS_dicionario_ados.csv</code>	O dicionário em planilha: 110 colunas (45 da silver e 65 do gold) com tipo, descrição, origem e exemplo
<code>requirements.txt</code>	As bibliotecas e versões (ver a página de bibliotecas)
<code>.vscode/settings.json</code>	Faz o VS Code usar o Python do <code>.venv</code> , ativar o ambiente no terminal e rodar cada notebook a partir da própria pasta
<code>.gitignore</code>	O que não vai para o GitHub: CSVs (menos o dicionário), <code>.dbc</code> e <code>.dbf</code> , arquivos do Power BI e compactados, <code>data/staging/</code> , <code>data/gold/</code> , caches do Python e do <code>pytest</code> e o <code>.venv</code>

POR QUE O .GITIGNORE É ASSIM

Arquivo pesado ou que cada pessoa gera na própria máquina não vai para o Git: a `staging` e o `gold` são refeitos com um comando, o `.venv` é de cada computador e o CSV da fato passa do limite de 100 MB do GitHub. As planilhas do IBGE (`.xls` e `.xlsx`) ficam no Git porque o pipeline precisa delas.

As convenções do código

CONVENÇÃO	COMO APARECE	POR QUÊ
Nomes em português	extrair_sih, transformar_cnes, df_populacao	O domínio (SUS) é brasileiro; o código lê como o negócio fala
snake_case em funções e variáveis	listar_arquivos_sih	Padrão do Python (PEP 8)
MAIÚSCULAS para constantes	COLUNAS_SIH, SAFRA_PADRAO	Deixa claro o que não muda durante a execução
Verbo no nome da função	extrair_, transformar_, processar_, montar_, gravar_, conferir_	O nome diz a etapa
"_" para uso interno	_padronizar_2022, _pct	Sinaliza que só é usada dentro do arquivo
Type hints	-> pd.DataFrame, -> list[Path]	Mostra o que entra e sai sem ler o corpo da função
pathlib em vez de texto	RAW_DIR / "sih"	Funciona no Windows e no Linux
Comentário curto por bloco	# Carrega e consolida os arquivos do SIH	Explica a intenção, não repete o código
Docstring no topo dos scripts	converter_dbc, promover_silver, gerar_modelo, testar_estimativa	Diz como usar sem precisar do README
Códigos como texto	zfill(6), dtype={"COD. MUNIC": str}	Não perder zeros à esquerda (CNES 0965324)
Erro explícito	raise FileNotFoundError, errors="raise", SystemExit("ERRO...")	Parar cedo com mensagem clara em vez de gerar dado errado
Rastreabilidade	coluna ARQUIVO_ORIGEM	Saber de qual arquivo mensal veio cada linha
Nome de arquivo diz o conteúdo	sih_multianual.parquet, test_staging_silver.py	Dá para achar as coisas sem abrir
Notebooks numerados	01_ a 06_	A ordem é a ordem do raciocínio

Como rodar tudo, na ordem

No Git Bash, na pasta raiz do projeto. Cada comando precisa terminar sem nenhuma linha de ERRO.

```
# 0. Só na primeira vez: criar e ativar o ambiente e instalar as bibliotecas
py -3.12 -m venv .venv
source .venv/Scripts/activate          # aparece (.venv) no começo da linha
pip install -r requirements.txt

# 1. Só quando entrar mês novo: .dbc baixado → .parquet
python -m src.converter_dbc

# 2. Refazer a staging (ou: python -m src.orchestration.prefect_flow)
python -m src.pipeline

# 3. Testar e promover staging → silver (só copia se passar)
python -m src.promover_silver

# 4. Opcional: rodar todos os testes de novo
python -m pytest

# 5. Gerar as 10 tabelas do painel, com a conferência
python gerar_modelo_bi_girosus.py

# 6. Opcional: provar os números da defasagem
python testar_estimativa_defasagem.py
```

COMANDO	O QUE TEM QUE APARECER
converter_dbc	“Nada a converter” ou um OK por .dbc novo
pipeline	3 linhas OK: SIH 2.286.755 linhas e 23 colunas, CNES 198.448 e 12, População 33.422 e 10
promover_silver	12 passed, 3 arquivos copiados, 3 passed e “Silver atualizada”
pytest	15 passed
gerar_modelo_bi_girosus	10 tabelas geradas, 10 linhas OK e “Todos os 223 numeros conferidos batem com o PDF”
testar_estimativa_defasagem	“OK: os 9 números do slide ... batem”

QUERO...	RODO
Só atualizar o Power BI com a silver atual	Passo 5
Incluir um mês novo do SIH ou do CNES	Baixar o .dbc → passos 1, 2, 3 e 5 (e atualizar os testes de cobertura)
Estudar os números	Notebook 06_girosus_ocupacao_leitos, “Run All”

SE O COMANDO NÃO MOSTRAR NADA

Sem o .venv ativo, o `python` do Windows pode chamar um atalho da Microsoft Store que não faz nada. Ative o ambiente ou use `.venv/Scripts/python.exe`. Avisos amarelos (DeprecationWarning) vêm das bibliotecas e não mudam nenhum número; só é problema se aparecer ERRO ou FAILED.

O que é feito nos dados, do DATASUS ao Power BI

Cada tratamento: o que é feito, por quê, onde está no código e quantas linhas afeta. Números da execução de 30/09/2026.

#	TRATAMENTO	POR QUÊ	ONDE	LINHAS AFETADAS
1	.dbc para .parquet	O .dbc é um formato compactado antigo. O Parquet é leve e guarda o tipo. Tudo entra como texto, sem mudar valor	src/converter_dbc.py (PySUS)	66 arquivos do SIH (2.286.755 registros) e 67 do CNES (198.448 linhas)
2	Datas e números no tipo certo	O arquivo chega todo em texto. Somar dias e valores e filtrar por data exigem o tipo certo	transform/sih.py (DT_INTER), transform/cnes.py (quantidades), gerar_modelo (DT_SAIDA, DIAS_PERM, VAL_TOT, UTI)	2.286.755 AIHs do SIH e 198.448 linhas do CNES
3	Ano e mês pelo nome do arquivo	Cada arquivo de leitos é de um mês (LTGO2401 = jan/2024); o nome garante o mês certo	transform/cnes.py	198.448 linhas do CNES
4	Zero à esquerda preservado	0965324 não pode virar 965324, senão não liga com as outras tabelas	leitura como texto no extract; zfill no gerar_modelo; .pq importa como Texto	165.522 AIHs com CNES começando em 0 (38.662 na safra 2025) e 7.043 linhas do CNES
5	Só AIH nova (IDENT = 1)	A AIH de continuação é a mesma internação cobrada em partes	gerar_modelo · carregar_sih()	2.286.755 → 2.235.030 (saem 51.725)
6	Safra 2025 pela competência	O painel olha o que foi apresentado ao SUS de jan a dez/2025	gerar_modelo · main()	2.235.030 → 457.403
7	Leito-dia, tempo típico, dias acima e grupo	Regras do produto: medir em dias, comparar com a mediana do procedimento, agrupar pelo CID	montar_fato_internacoes() e classificar_grupo()	457.403 internações; 163.861 com dias acima do típico
8	UTI e acompanhante do bruto	VAL_UTI e DIAR_ACOM ainda não estão na silver. Sem o bruto, o script para em vez de gerar coluna vazia	carregar_complementos_raw()	457.403 ligadas; 41.880 com valor de UTI e 236.063 com diária de acompanhante
9	Testes de vazios e duplicados	Garantir a base antes de qualquer análise; a silver só muda se passar	tests/ (15) e promover_silver.py	2.286.755 (SIH), 198.448 (CNES) e 33.422 (população)
10	Trava contra células vazias	Nenhuma tabela do painel pode sair com vazio, como já aconteceu com nome_hospital	conferir_saida()	As 10 tabelas; hoje 0 células vazias

O que ainda pode melhorar

#	O QUÊ	ONDE	IMPACTO	SUGESTÃO
1	O download dos .dbc ainda é manual	site do DATASUS	Trabalho manual todo mês	Automatizar o download no converter_dbc.py
2	Notebooks 01, 02 e 04 leem os .dbc	notebooks/	Não rodam num clone novo	Ler os .parquet de data/raw
3	O notebook 04 grava na silver com DT_INTER como texto	notebooks/04	Se rodar a última célula, o teste staging × silver falha	Não rodar essa célula ou remover a gravação
4	VAL_UTI e DIAR_ACOM não estão na silver	extract/sih.py	O script precisa do data/raw/sih	Incluir as duas em COLUNAS_SIH e promover
5	Regras repetidas no notebook 06 e no script	notebooks/06 e gerar_modelo	Mudar uma regra exige mudar nos dois	Mover as regras para um módulo e importar nos dois
6	Testes de cobertura com o último mês fixo	test_sih.py, test_cnes.py	Todo mês novo exige ajustar 66 e 67	Calcular o esperado a partir de config.py

Como falar disso numa avaliação

“O projeto tem as dívidas mapeadas. Já resolvemos a promoção automática para a silver, as mensagens do pipeline, a conversão dos .dbc, a trava contra campos vazios e os links da documentação. As que restam são de organização e não mudam nenhum número do painel.”

Glossário técnico

TERMO	EM PORTUGUÊS CLARO
ETL	Extract, Transform, Load: extrair, transformar e carregar. É o que a pasta src/ faz
Pipeline	Sequência automática de etapas: aqui, extrair → transformar → gravar
Camada (raw, staging, silver, gold)	Nível de tratamento do dado. Quanto mais perto do gold, mais pronto para uso
Arquitetura medalhão	Nome comum para organizar dados em camadas bronze (bruto), silver (limpo) e gold (pronto)
Módulo	Um arquivo .py que pode ser importado por outro (ex.: src.extract.sih)
Parquet	Formato de arquivo compacto que guarda o tipo de cada coluna
DataFrame	A tabela do pandas: linhas e colunas na memória
Fixture	No pytest, um dado preparado uma vez e entregue aos testes que pedem por ele
Teste de regressão	Confere se algo mudou sem ninguém querer (staging × silver)
Orquestração (Prefect)	Ferramenta que executa, registra e agenda as etapas do pipeline
Ambiente virtual (.venv)	Uma instalação de Python só do projeto, com as versões do requirements
AIH	Autorização de Internação Hospitalar: o registro de cada internação paga pelo SUS
Competência	Mês em que o hospital apresentou a internação para pagamento; organiza os arquivos do DATASUS
Safra	Ano da competência. O painel usa a safra 2025
Leito-dia	Um paciente ocupando um leito por um dia
Defasagem	O atraso entre a alta do paciente e a chegada da conta no SIH
Modelo estrela	Tabela fato no centro ligada a dimensões; é como o gold está organizado para o Power BI
Fato / dimensão	Fato = o que aconteceu (internação). Dimensão = como filtrar (tempo, hospital, doença)
Power Query (.pq)	A linguagem de importação do Power BI; as consultas prontas ficam em power_query/
Type hint	Anotação do tipo que entra e sai de uma função (-> pd.DataFrame)